

InternNova – Week 3 Assignment

SQL & Excel

SQL Database and Sample Data Setup-

```
CREATE DATABASE IF NOT EXISTS
```

```
internnova_week3_assignment;
```

```
USE internnova_week3_assignment;
```

```
DROP TABLE IF EXISTS Employees;
```

```
DROP TABLE IF EXISTS Departments;
```

```
DROP TABLE IF EXISTS Products;
```

```
CREATE TABLE Departments (
```

```
    department_id INT PRIMARY KEY,
```

```
    department_name VARCHAR(50) NOT NULL
```

```
);
```

```
CREATE TABLE Employees (
```

```
    employee_id INT PRIMARY KEY,
```

```
    employee_name VARCHAR(50) NOT NULL,
```

```
    salary DECIMAL(10,2) NOT NULL,
```

```
    department_id INT NOT NULL,
```

```
    hire_date DATE NOT NULL,
```

```
    FOREIGN KEY (department_id) REFERENCES
```

```
Departments(department_id)
```

```
);
```

```
CREATE TABLE Products (
```

```
    product_id INT PRIMARY KEY,
```

```
product_name VARCHAR(80) NOT NULL,  
category VARCHAR(40) NOT NULL,  
unit_price DECIMAL(10,2) NOT NULL  
);
```

```
INSERT INTO Departments (department_id, department_name)  
VALUES  
(1, 'Sales'),  
(2, 'IT'),  
(3, 'HR'),  
(4, 'Marketing'),  
(5, 'Finance');
```

```
INSERT INTO Employees (employee_id, employee_name, salary,  
department_id, hire_date) VALUES  
(101, 'Alice Sharma', 68000.00, 1, '2022-02-15'),  
(102, 'Bob Mehta', 52000.00, 2, '2023-06-10'),  
(103, 'Carol Singh', 91000.00, 1, '2021-08-21'),  
(104, 'David Kumar', 60000.00, 3, '2024-01-12'),  
(105, 'Eva Kapoor', 75000.00, 2, '2022-11-03'),  
(106, 'Farhan Ali', 47000.00, 4, '2024-04-18'),  
(107, 'Grace Nair', 83000.00, 1, '2023-03-27'),  
(108, 'Hari Rao', 56000.00, 2, '2025-01-09');
```

```
INSERT INTO Products (product_id, product_name, category,  
unit_price) VALUES  
(201, 'Laptop Pro 14', 'Electronics', 850.00),  
(202, 'Wireless Mouse', 'Accessories', 25.00),  
(203, 'Mechanical Keyboard', 'Accessories', 75.00),  
(204, '27-inch Monitor', 'Electronics', 320.00),  
(205, 'USB-C Hub', 'Accessories', 45.00),
```

(206, 'Office Chair', 'Furniture', 220.00),
(207, 'Standing Desk', 'Furniture', 480.00),
(208, 'Noise-Cancel Headset', 'Electronics', 110.00);

Database Table Screenshots

select * from employees;

	employee_id	employee_name	salary	department_id	hire_date	
	101	Alice Sharma	68000.00	1	2022-02-15	
	102	Bob Mehta	52000.00	2	2023-06-10	
	103	Carol Singh	91000.00	1	2021-08-21	
	104	David Kumar	60000.00	3	2024-01-12	
	105	Eva Kapoor	75000.00	2	2022-11-03	
	106	Farhan Ali	47000.00	4	2024-04-18	
	107	Grace Nair	83000.00	1	2023-03-27	
	108	Hari Rao	56000.00	2	2025-01-09	
	NULL	NULL	NULL	NULL	NULL	

select * from departments;

	department_id	department_name
	1	Sales
	2	IT
	3	HR
	4	Marketing
	5	Finance
	NULL	NULL

```
select * from products;
```

	product_id	product_name	category	unit_price	
	201	Laptop Pro 14	Electronics	850.00	
	202	Wireless Mouse	Accessories	25.00	
	203	Mechanical Keyboard	Accessories	75.00	
	204	27-inch Monitor	Electronics	320.00	
	205	USB-C Hub	Accessories	45.00	
	206	Office Chair	Furniture	220.00	
	207	Standing Desk	Furniture	480.00	
	208	Noise-Cancel Headset	Electronics	110.00	
	NULL	NULL	NULL	NULL	

Task 1: Introduction to Databases & SELECT Statement

Marks: 10

1. Brief Explanation

A database stores related information in structured tables so records can be added, searched and analysed efficiently. For this task, the Employees table is used. SELECT * returns every column and row, while selecting individual columns keeps the result focused. Aliases make the result headings easier to read.

2. SQL Syntax with Program Output

-- 1A. Display all records.

```
SELECT * FROM Employees;
```

	employee_id	employee_name	salary	department_id	hire_date	
	101	Alice Sharma	68000.00	1	2022-02-15	
	102	Bob Mehta	52000.00	2	2023-06-10	
	103	Carol Singh	91000.00	1	2021-08-21	
	104	David Kumar	60000.00	3	2024-01-12	
	105	Eva Kapoor	75000.00	2	2022-11-03	
	106	Farhan Ali	47000.00	4	2024-04-18	
	107	Grace Nair	83000.00	1	2023-03-27	
	108	Hari Rao	56000.00	2	2025-01-09	
	NULL	NULL	NULL	NULL	NULL	

-- 1B. Select specific columns and use aliases.

```
SELECT employee_name AS Employee,  
       salary AS Annual_Salary,  
       department_id AS Department_ID  
FROM Employees  
ORDER BY employee_id;
```

	Employee	Annual_Salary	Department_ID	
	Alice Sharma	68000.00	1	
	Bob Mehta	52000.00	2	
	Carol Singh	91000.00	1	
	David Kumar	60000.00	3	
	Eva Kapoor	75000.00	2	
	Farhan Ali	47000.00	4	
	Grace Nair	83000.00	1	
	Hari Rao	56000.00	2	

Task 2: WHERE, ORDER BY & Aggregate Functions

Marks: 15

1. Brief Explanation

The WHERE clause filters rows before the result is returned. The comparison operator $\geq$ keeps employees earning at least 60,000. ORDER BY salary DESC sorts the qualifying rows from highest to lowest salary. The aggregate query then calculates the number of employees, total salary, average salary, minimum salary and maximum salary.

2. SQL Syntax with Program Output

-- 2A. Filter employees earning at least 60,000 and sort from highest to lowest.

```
SELECT employee_id, employee_name, salary, department_id
FROM Employees
WHERE salary >= 60000
ORDER BY salary DESC;
```

	employee_id	employee_name	salary	department_id	
	103	Carol Singh	91000.00	1	
	107	Grace Nair	83000.00	1	
	105	Eva Kapoor	75000.00	2	
	101	Alice Sharma	68000.00	1	
	104	David Kumar	60000.00	3	
	NULL	NULL	NULL	NULL	

-- 2B. Use COUNT, SUM, AVG, MIN and MAX.

```
SELECT COUNT(*) AS Employee_Count,
SUM(salary) AS Total_Salary,
ROUND(AVG(salary), 2) AS Average_Salary,
MIN(salary) AS Minimum_Salary,
MAX(salary) AS Maximum_Salary
FROM Employees;
```

	Employee_Count	Total_Salary	Average_Salary	Minimum_Salary	Maximum_Salary	
	8	532000.00	66500.00	47000.00	91000.00	

Task 3: GROUP BY & HAVING

Marks: 10

1. Brief Explanation

GROUP BY creates one group for each department. COUNT() measures the number of employees in each group and AVG() calculates the department-level average salary. HAVING is then used to keep only groups whose average salary is above 60,000. This differs from WHERE because HAVING filters grouped results after aggregation.

2. SQL Syntax with Program Output

-- Show departments whose average salary is above 60,000.

```
SELECT d.department_name AS Department,  
       COUNT(e.employee_id) AS Employee_Count,  
       ROUND(AVG(e.salary), 2) AS Average_Salary  
FROM Employees e  
JOIN Departments d  
  ON e.department_id = d.department_id  
GROUP BY d.department_id, d.department_name  
HAVING AVG(e.salary) > 60000  
ORDER BY Average_Salary DESC;
```

	Department	Employee_Count	Average_Salary	
	Sales	3	80666.67	
	IT	3	61000.00	

Task 4: SQL JOINS

Marks: 15

1. Brief Explanation

Joins combine rows from related tables using a common column. The shared key here is department_id. INNER JOIN returns only matching department-employee pairs. LEFT JOIN keeps every department from the left table and fills unmatched employee fields with NULL. RIGHT JOIN keeps every department from the right table, which also makes the department with no employee visible.

2. SQL Syntax with Program Output

-- 4A. INNER JOIN: only departments with matching employees.

```
SELECT d.department_name AS Department,  
       e.employee_name AS Employee,  
       e.salary AS Salary  
FROM Employees e  
INNER JOIN Departments d  
ON e.department_id = d.department_id  
ORDER BY d.department_id, e.employee_id;
```

	Department	Employee	Salary	
	Sales	Alice Sharma	68000.00	
	Sales	Carol Singh	91000.00	
	Sales	Grace Nair	83000.00	
	IT	Bob Mehta	52000.00	
	IT	Eva Kapoor	75000.00	
	IT	Hari Rao	56000.00	
	HR	David Kumar	60000.00	
	Marketing	Farhan Ali	47000.00	

-- 4B. LEFT JOIN: keep every department and show matching employees when present.

```
SELECT d.department_name AS Department,  
       e.employee_name AS Employee,  
       e.salary AS Salary  
FROM Departments d  
LEFT JOIN Employees e  
ON d.department_id = e.department_id  
ORDER BY d.department_id, e.employee_id;
```

	Department	Employee	Salary	
	Sales	Alice Sharma	68000.00	
	Sales	Carol Singh	91000.00	
	Sales	Grace Nair	83000.00	
	IT	Bob Mehta	52000.00	
	IT	Eva Kapoor	75000.00	
	IT	Hari Rao	56000.00	
	HR	David Kumar	60000.00	
	Marketing	Farhan Ali	47000.00	
	Finance	NULL	NULL	

-- 4C. RIGHT JOIN: keep every department from the right-hand table.

SELECT d.department_name AS Department,

e.employee_name AS Employee,

e.salary AS Salary

FROM Employees e

RIGHT JOIN Departments d

ON e.department_id = d.department_id

ORDER BY d.department_id, e.employee_id;

	Department	Employee	Salary	
	Sales	Alice Sharma	68000.00	
	Sales	Carol Singh	91000.00	
	Sales	Grace Nair	83000.00	
	IT	Bob Mehta	52000.00	
	IT	Eva Kapoor	75000.00	
	IT	Hari Rao	56000.00	
	HR	David Kumar	60000.00	
	Marketing	Farhan Ali	47000.00	
	Finance	NULL	NULL	

Task 5: SQL Subqueries

Marks: 10

1. Brief Explanation

A subquery is a query nested inside another query. The first example calculates the average employee salary and then keeps employees whose salary is higher than that average. The second example calculates the average product price and returns products priced above that value.

2. SQL Syntax with Program Output

-- 5A. Employees earning more than the average employee salary.

```
SELECT employee_name AS Employee,  
       salary AS Salary  
FROM Employees  
WHERE salary > (SELECT AVG(salary) FROM Employees)  
ORDER BY salary DESC;
```

Employee	Salary
Carol Singh	91000.00
Grace Nair	83000.00
Eva Kapoor	75000.00
Alice Sharma	68000.00

-- 5B. Products priced above the average product price.

```
SELECT product_name AS Product,  
       unit_price AS Unit_Price  
FROM Products  
WHERE unit_price > (SELECT AVG(unit_price) FROM Products)  
ORDER BY unit_price DESC;
```

Product	Unit_Price
Laptop Pro 14	850.00
Standing Desk	480.00
27-inch Monitor	320.00

Task 6: Data Formatting, Sorting & Filtering

Marks: 10

1. Brief Explanation

The Sales_Data tab holds all 120 orders. I bolded the headers and gave them a navy background, then applied formats suited to each column: readable dates, currency for sales, percentages for discounts, and profit with negative values in red brackets. The rows are sorted by order date, the header row is frozen so it stays visible while scrolling, and AutoFilter is turned on. The T6_Filtered_View tab shows the filter in use. It keeps only Region = West and Category = Technology, which leaves 10 of the 120 records visible. Its cells link back to Sales_Data, so the values stay in sync.

2. Excel Record

Order ID	Order Date	Ship Mode	Customer ID	Customer Name	Segment	Region	Product ID	Category	Sub-Category	Sales	Quantity	Discount	Profit	Loss Flag	Filter applied: Region = West AND Category = Technology
ORD-2016-0006	16-Jan-2016	Same Day	CU-1001	Aarav Mehta	Consumer	West	TEC-MA-01	Technology	Machines	\$4,495.00	5	0%	\$635.48	Profit	
ORD-2016-0010	26-Jan-2016	Second Class	CU-1010	Saanvi Joshi	Consumer	West	TEC-AC-01	Technology	Accessories	\$314.93	7	0%	\$20.71	Profit	
ORD-2016-0012	06-Feb-2016	Standard Class	CU-1005	Kabir Singh	Corporate	West	TEC-AC-01	Technology	Accessories	\$26.99	1	40%	(\$3.30)	Loss	
ORD-2016-0023	16-Mar-2016	Second Class	CU-1010	Saanvi Joshi	Consumer	West	TEC-AC-01	Technology	Accessories	\$283.44	7	10%	\$46.17	Profit	
ORD-2016-0032	07-Apr-2016	Standard Class	CU-1010	Saanvi Joshi	Consumer	West	TEC-AC-01	Technology	Accessories	\$31.49	1	30%	\$0.28	Profit	
ORD-2016-0062	30-Jun-2016	Standard Class	CU-1005	Kabir Singh	Corporate	West	TEC-PH-02	Technology	Phones	\$116.99	1	10%	(\$6.17)	Loss	
ORD-2016-0090	14-Sep-2016	Standard Class	CU-1005	Kabir Singh	Corporate	West	TEC-MA-01	Technology	Machines	\$3,146.50	5	30%	\$0.35	Profit	
ORD-2016-0102	24-Oct-2016	Standard Class	CU-1010	Saanvi Joshi	Consumer	West	TEC-PH-02	Technology	Phones	\$779.94	6	0%	\$22.81	Profit	
ORD-2016-0110	18-Nov-2016	Standard Class	CU-1014	Myra Chawla	Home Office	West	TEC-PH-01	Technology	Phones	\$5,841.00	9	0%	\$932.26	Profit	
ORD-2016-0111	20-Nov-2016	Standard Class	CU-1019	Dhruv Agarwal	Corporate	West	TEC-PH-02	Technology	Phones	\$155.99	2	40%	(\$16.00)	Loss	

Task 7: Conditional Formatting & Excel Functions

Marks: 15

1. Brief Explanation

Four conditional formatting rules colour the dataset: negative profit in red, sales of 1,000 or more in green, discounts of 30% or more in yellow, and rows labelled "Loss" in red. These make loss-making and heavily discounted orders easy to spot. On the T7 tab, IF labels each order as Profit or Loss and gives an overall verdict. COUNTIF

counts the loss-making orders (42), and a second COUNTIF cross-checks that number. SUMIF adds up values that meet a condition, such as Technology sales (59,860.19) and Furniture profit (−365.40). A category summary combines all three functions and marks Furniture as loss-making.

2. Excel Record

	A	B	C	D	E
1	Task 7: Conditional Formatting and Excel Functions				
2	Source data: Sales_Data rows 2 to 121. Every result below is a live formula.				
3					
4	A. Conditional formatting rules (applied on Sales_Data)				
5	Column	Rule	Format applied	Why it is useful	
6	Profit (N)	Value is less than 0	Light red fill, dark red text	Spots loss-making orders at a glance.	
7	Sales (K)	Value is 1,000 or more	Light green fill, dark green text	Highlights high-value orders.	
8	Discount (M)	Value is 30% or more	Light yellow fill, brown text	Flags heavy discounts, which can push an order into a loss.	
9	Loss Flag (O)	Text equals Loss	Light red fill, dark red text	Highlights rows that the IF formula labelled Loss.	
10					
11	B. Functions: IF, COUNTIF and SUMIF				
12	Function	Result	What it answers	Purpose of the function	Check
13	IF	Profitable overall	Is total profit positive?	IF returns one value when a condition is TRUE and another when it is FALSE. Used here for the overall verdict and for the per-order Loss Flag in Sales_Data column O.	
14	COUNTIF	42	How many orders made a loss?	COUNTIF counts the cells in a range that meet one condition. Used here to count negative profit values.	
15	COUNTIF	42	Cross-check: how many rows are labelled Loss?	Counts text matches. It should equal the result above, so the IF formulas and the COUNTIF agree.	Matches
16	SUMIF	\$59,860.19	Total sales for the Technology category	SUMIF adds the values in one column (Sales) only where a condition in another column (Category) is met.	
17	SUMIF	(\$365.40)	Total profit for the Furniture category	A second SUMIF, this time adding Profit. A negative result means the category lost money overall.	
8					
9	C. Category summary combining COUNTIF, SUMIF and IF				
0	Category	Orders (COUNTIF)	Total Sales (SUMIF)	Total Profit (SUMIF)	Status (IF)
1	Furniture	42	\$28,386.71	(\$365.40)	Loss-making
2	Office Supplies	38	\$4,033.29	\$580.74	Profitable
3	Technology	40	\$59,860.19	\$5,067.60	Profitable
4	Total	120	\$92,280.19	\$5,282.94	Profitable
5					
6					

Task 8: VLOOKUP and XLOOKUP

Marks: 10

1. Brief Explanation

Product_Master is the second table, linked to Sales_Data through Product ID. On the T8 tab, VLOOKUP and XLOOKUP each return the product name and unit price for 12 sample orders, and a Match column confirms that both give the same results. A test

row with an unknown Product ID shows "Not found" from both functions instead of an error. The reverse lookup section shows the key difference: finding a Product ID from a Product Name. VLOOKUP can't return a column to the left of its lookup column, but XLOOKUP can. A comparison table summarises the differences in syntax, default matching, and handling of missing values.

2. Excel Record

Task 8: VLOOKUP and XLOOKUP

Product_Master (table 2) is looked up using Product ID from Sales_Data (table 1). The two tables are related through Product ID.

Part A: Retrieve product details for sample orders

Order ID	Product ID	VLOOKUP: Product Name	VLOOKUP: Unit Price	XLOOKUP: Product Name	XLOOKUP: Unit Price	Results match?
ORD-2016-0001	OFS-PA-01	Copier Paper A4 (Ream)	\$9.60	Copier Paper A4 (Ream)	\$9.60	Match
ORD-2016-0002	OFS-ST-02	Plastic Storage Bins (Set of 4)	\$38.20	Plastic Storage Bins (Set of 4)	\$38.20	Match
ORD-2016-0003	TEC-MA-01	Multifunction Laser Printer	\$899.00	Multifunction Laser Printer	\$899.00	Match
ORD-2016-0004	TEC-AC-01	Wireless Keyboard and Mouse	\$44.99	Wireless Keyboard and Mouse	\$44.99	Match
ORD-2016-0005	FUR-BK-02	Glass-Door Cabinet	\$365.00	Glass-Door Cabinet	\$365.00	Match
ORD-2016-0006	TEC-MA-01	Multifunction Laser Printer	\$899.00	Multifunction Laser Printer	\$899.00	Match
ORD-2016-0007	FUR-TB-01	Conference Table 8-Seater	\$1,180.00	Conference Table 8-Seater	\$1,180.00	Match
ORD-2016-0008	TEC-AC-01	Wireless Keyboard and Mouse	\$44.99	Wireless Keyboard and Mouse	\$44.99	Match
ORD-2016-0009	TEC-AC-01	Wireless Keyboard and Mouse	\$44.99	Wireless Keyboard and Mouse	\$44.99	Match
ORD-2016-0010	TEC-AC-01	Wireless Keyboard and Mouse	\$44.99	Wireless Keyboard and Mouse	\$44.99	Match
ORD-2016-0011	OFS-ST-02	Plastic Storage Bins (Set of 4)	\$38.20	Plastic Storage Bins (Set of 4)	\$38.20	Match
ORD-2016-0012	TEC-AC-01	Wireless Keyboard and Mouse	\$44.99	Wireless Keyboard and Mouse	\$44.99	Match
TEST-01	FAKE-999	Not found	Not found	Not found	Not found	Match

Row 18 uses an ID that is not in Product_Master (yellow cells are inputs). Both functions return "Not found" instead of an error.

Part B: Reverse lookup, finding Product ID from Product Name

Product name (input):

Ergonomic Mesh Chair

Method	Product ID returned	Notes
VLOOKUP	Not possible	VLOOKUP only returns columns to the right of the lookup column. Product ID sits to the left of Product Name, so VLOOKUP cannot return it.
INDEX/MATCH	FUR-CH-02	Works around the VLOOKUP limit, but needs two functions.
XLOOKUP	FUR-CH-02	One function. It returns the column to the left directly.

Part C: VLOOKUP vs XLOOKUP at a glance

Feature	VLOOKUP	XLOOKUP
Syntax	VLOOKUP(lookup, table, col_index, [range_lookup])	XLOOKUP(lookup, lookup_array, return_array, [if_not_found], ...)
Direction	Returns only columns to the right of the key column	Returns any column, to the left or to the right
Column reference	A column number (2, 5, ...) that breaks if columns are inserted	A direct range reference, so no counting is needed
Default match	Approximate match if range_lookup is left out (a common source of errors)	Exact match by default
Missing value	Shows an N/A error unless wrapped in IFERROR	Built-in if_not_found argument
Availability	Every Excel version	Excel 2021, Excel 365 and Google Sheets

Task 9: Pivot Table and Chart

Marks: 5

1. Brief Explanation

Table 1 is a pivot-style summary of sales by category and region, built with SUMIFS formulas. It is not a native Excel PivotTable, but the tab includes steps to create one in about a minute. Table 2 shows sales, profit and profit margin for each category.

Technology has the highest sales (59,860) *and profit* (5,068), but Office Supplies has the best margin (about 14%), and Furniture loses money overall (margin about -1.3%). The clustered column chart compares sales across regions for each category.

Technology leads in Central, East and West, while Furniture leads in North and South. The written explanation on the sheet points out that the category with the most sales is not the most profitable relative to its sales.

2. Excel Record

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

A

B

C

D

E

F

G

H

I

J

K

L

M

N

O

P

Q

R

S

T

Task 9: Pivot Table and Chart

Source: Sales_Data. Tables below use SUMIFS and SUMIF formulas, so they update when the data changes.

Table 1: Sales by Category and Region (pivot-style summary)

Category	Central	East	North	South	West	Grand Total
Furniture	\$3,059	\$3,239	\$5,987	\$7,028	\$9,074	\$28,387
Office Supplies	\$1,138	\$312	\$293	\$1,347	\$944	\$4,033
Technology	\$21,109	\$15,068	\$3,331	\$5,160	\$15,192	\$59,860
Grand Total	\$25,306	\$18,619	\$9,610	\$13,535	\$25,210	\$92,280

Table 2: Sales, profit and profit margin by Category

Category	Total Sales	Total Profit	Profit Margin
Furniture	\$28,387	(\$365.40)	-1.3%
Office Supplies	\$4,033	\$580.74	14.4%
Technology	\$59,860	\$5,067.60	8.5%
Total	\$92,280	\$5,282.94	5.7%

Chart explanation

The clustered column chart compares sales for each category across the five regions. Technology has the largest bars in Central, East and West, with Central the highest at about \$21,109. Furniture leads in North and South, and its biggest region is West at about \$9,074. Office Supplies is the smallest category in every region. The chart measures sales volume only. Table 2 shows that Furniture lost money overall (margin about -1.3%), while Office Supplies earned about 14.4% and Technology about 8.5%. In dollar terms Technology is also the most profitable category, but its margin is below Office Supplies, and Furniture loses money overall even though it leads sales in North and South.

How to make a native PivotTable in Excel (optional, takes about one minute)

Click any cell in Sales_Data, then go to Insert > PivotTable and choose OK. Drag Category into Rows, Region into Columns, and Sales into Values (Sum). Table 1 above shows the same figures.

Sales by Category and Region

CentralEastNorthSouthWest

Category	Central	East	North	South	West
Furniture	\$3,059	\$3,239	\$5,987	\$7,028	\$9,074
Office Supplies	\$1,138	\$312	\$293	\$1,347	\$944
Technology	\$21,109	\$15,068	\$3,331	\$5,160	\$15,192

